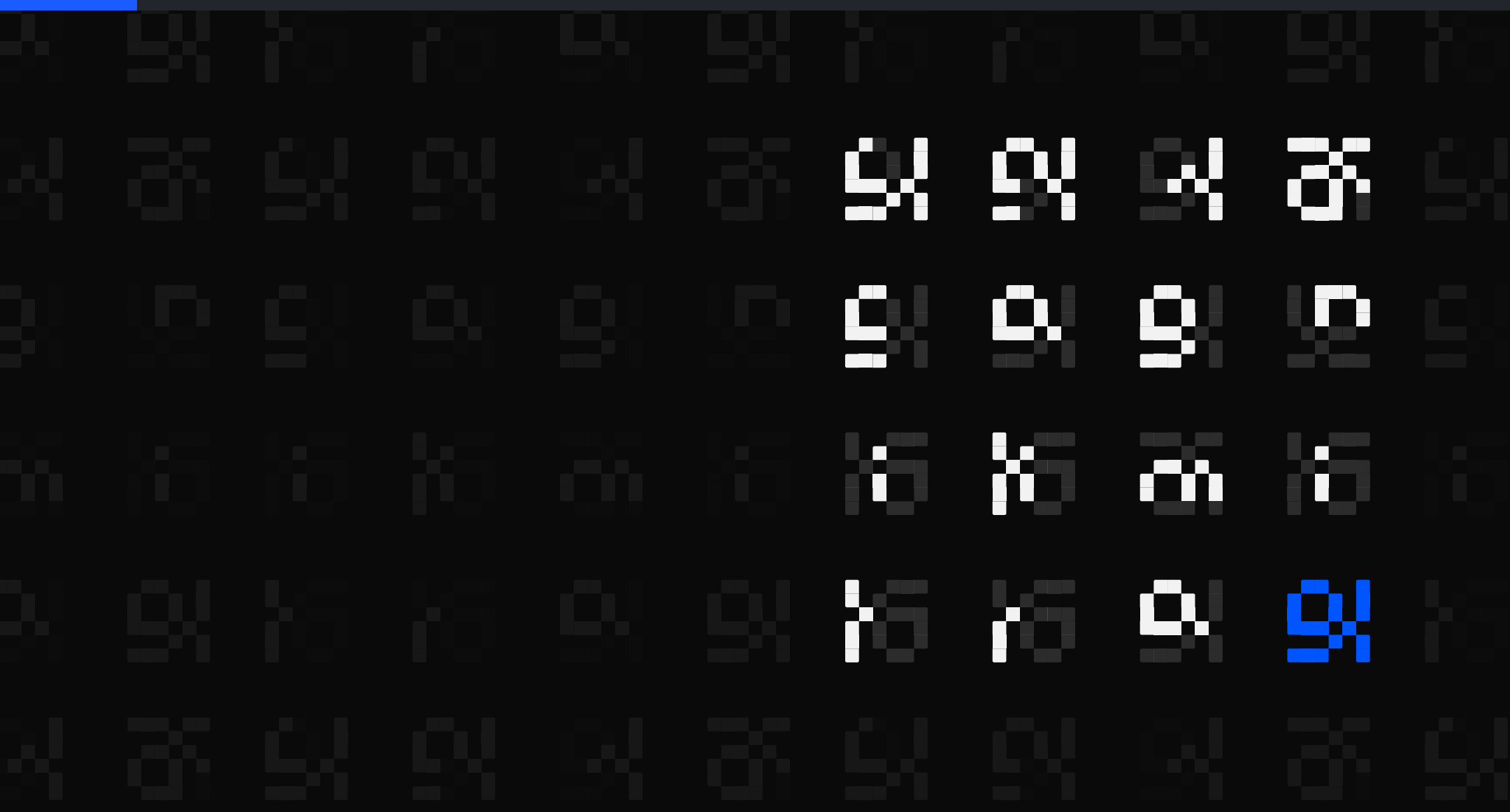




The Brilliant Employee · 02

A security alarm named a file that never existed

The Incident Files. The address was invented, the credential type was not. The real tokens were sitting in plaintext in my own chat logs. The pattern matched 47 of 15,110 transcript files, and reading them left two real credentials.

sgnk.ai/brilliant ↗

Sagnik Mitra

Why this exists

sgnk is a one-person studio. A language model writes a large share of the production code. You do not need your own system to get answers out of a model. You need one to find out when the answers were wrong, because wrong work reads exactly like right work and no prompt fixes that.

Three things from that week, none visible from inside the tool: a sizing step ignored on 796 of 816 tasks, two fixes I had written up as done that were never made, and a detector whose 1,112 catches were mostly one test string of my own.

THE SYSTEM, AS OF 6 AUGUST 2026, 16:43Z	
rules written	11 June, 56 days ago
ledger running	27 June, 40 days
tasks logged	3,063, across 3,562 rows
controls on tool calls	6 registered, 3 of them armed

You can rent the model. You cannot rent the part that tells you it was wrong.

Five parts, and where today sits

Five parts sit around the model. What changes from one piece of writing to the next is which of them is under the microscope.

the Gate	sizes a job before a model is picked
the Gambler	recommends which model gets the job
the Rails	refuse the irreversible, before it runs
the Judge	grades finished work pass or fail
the Ledger	at least one append-only line per task

Listed in the order they act on a task, not the order they were built.

Today cuts across more than one of them, so no row is marked. Nothing below is assumed knowledge; everything this post needs, it explains.

ASIDE. Only the rails can refuse. The gate advises, the router recommends, the judge grades after the fact, and the ledger only records. Four of the five observe; one intervenes.

This page is the map. Nothing here needs another post to make sense.

Detail arrives as fast as truth does

One of my automation system's audit subagents came back from a repository sweep with a specific, confident finding: leaked personal access tokens, sitting in testing/workspaces.json.

Plain words: PERSONAL ACCESS TOKEN. a password-shaped string that lets software act as you on GitHub. Whoever holds it is you, as far as the platform can tell.

WHAT THE FINDING CARRIED	
the path named	testing/workspaces.json
the credential named	personal access token
the stakes	whoever holds it is you

This is exactly the class of alarm you cannot sit on, and exactly the class you most want to forward untouched, which is the trap. Specific claims feel pre-verified. They are not.

Specificity is not evidence. It is just detail.

The named file never existed

Before repeating the finding anywhere, I went to the file. There is no testing/workspaces.json. Not moved, not renamed, not deleted in some recent commit. The path had never existed.

EXIT ONE

do

**close it as a
hallucination**

EXIT TWO

do

**soften it to a
possible leak**

Both exits end the search at exactly the moment it should widen. They are the same mistake in opposite directions, treating a subordinate's alarm as a verdict rather than a hypothesis.

I nearly took the first one. An alarm with a wrong address reads like a false alarm. It is not the same thing.

ASIDE. The address was never the load-bearing part of the alarm. The credential type was.

A wrong alarm can still be pointing at a real fire.

Widen from the path to the pattern

- 1 Alarm raised: leaked tokens at a specific named path
- 2 Named path checked live: the file does not exist
- 3 Search widened from the path to the pattern: token-shaped strings, anywhere

```
# the alarm named a path
$ find . -path "*testing/workspaces.json"
$

# widen from path to pattern
$ grep -rIE 'gh[pousr]_[A-Za-z0-9]{20,}' \
  ~/.claude/projects | wc -l
(a two-digit number, and rising)
```

The pivot is step three. Stop auditing the claim and start auditing the world.

Verify the claim. Then search wider than the pointer.

The leak was in my chat logs

Every session my automation system runs is written to a log file on my own disk, line by line, as it happens. The vendor's own documentation calls that file plaintext, and the same document says known key and token patterns are redacted before a shared session is uploaded. The redactor runs on the copy that leaves.

WHERE THE ALARM POINTED

```
path
testing/workspaces.json
exists
no
tokens found
0
```

WHERE THE FIRE WAS

```
path
~/.claude/projects/**/*.jsonl
what
his own session
transcripts
tokens found
GitHub-token-shaped
strings, plaintext
```

ASIDE. A public request to scrub the file that stays was filed against the claude-code repository and closed as not planned. A decision, not a blind spot.

The most dangerous file on the machine is the one that writes itself.

The auditor's confidence tells you nothing

The auditor is a language model too. It saw the shape of a plausible leak and produced a plausible location for it, with the same fluent confidence it produces everything. I cannot interview him, and I cannot tell his right days from his wrong days by tone.

Alarm

what the subordinate says

Hypothesis

what the workplace records it as

Fact

what survives a live check

So the workplace demotes every claim one level on arrival. Alarms enter as hypotheses by policy. Not because he is usually wrong, but because his confidence carries no information either way.

Confidence is a voice setting. Verification is a fact.

The lesson lives in the rulebook, not the worker

LEARNED RULE #2, STANDING RULEBOOK	
seeded	2026-06-27
trigger	any security or factual claim from a subagent
step one	verify it live before repeating it as fact
step two	attribute anything you could not verify
symmetry	never dismiss on a wrong pointer, widen the search
Append-only. The incident happened once. The rule loads every session.	

The rulebook is the actual workplace. He reads it at the start of every session, so the cost of the incident was paid once and the lesson compounds daily. That is the trade this whole series is about: the reliability lives in the paper around the employee, not in the employee.

An incident you survived is worth exactly what you wrote down.

Count distinct strings, not grep hits

- 1 Never repeat it as fact before a live check
- 2 Never dismiss it on a wrong pointer. Wrong address, real fire is a common shape
- 3 Widen from the named location to the pattern

47 of 15,110 files

matched the pattern

163 occurrences

of 20 distinct strings

7 at 40 characters

the length a real one is

Two real credentials

what reading the 7 left

ASIDE. Re-derived 2026-08-04 22:54Z, pattern `gh[pousr]_` then 20 or more characters.

Alarms are hypotheses. Fires are facts.
A live check is the only converter.



The scrubber runs on the copy it sends. Nobody is going to clean the copy it keeps, so run the pattern on your own machine and count distinct strings rather than hits.

sgnk.ai/brilliant ↗

Sagnik Mitra

I build AI systems and the machinery that keeps them honest: gates, ledgers, judges, and rails. Product design for years, engineering the whole time. This series is what the building actually taught me.

sgnk.ai · in/sagnikmitra · github.com/sagnikmitra